

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Slot-c

20

A

Course Code:	CSA1402	Course Title:	COMPILER DESIGN
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING
Weightage:	40% of CIA		
CO1 Statement:	Apply lexical analysis for token specification and recognition using LEX tool. (BL3)		
Student Name:	K. Deepthi	Reg. No.:	192525343
Section / Batch:		Date:	16/07/26

Code of Conduct

I K. Deepthi (192525343) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: 

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	4/4	3/4	4/4	4/4	3/4	91.25
2	4/4	3/4	3/4	3/4	4/4	83.75
3	3/4	4/4	4/4	4/4	3/4	91.25
4	4/4	4/4	3/4	3/4	3/4	85
5	3/4	3/4	4/4	4/4	4/4	90
OVERALL RUBRICS VALUE						
	/4	/4	/4	/4	/4	88.25
	/20	/20	/25	/20	/15	/100

Type

Identified

Assignment operator

Left Parenthesis

Identified

Multiplication operator

Identified

Right Parenthesis

Addition operator

Left Parenthesis

~~Identified~~

Multiplication operator

~~Identified~~

Right Parenthesis

Multiplication operator

Integer constant.

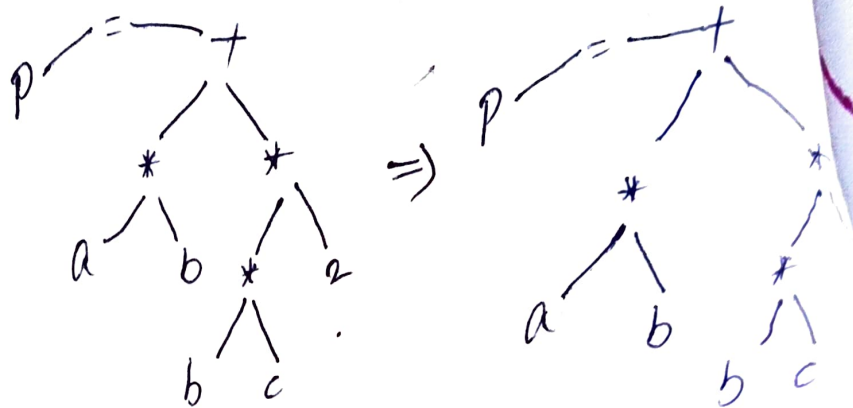
$$p = (a * b) + (b * c) * 2$$

a) Tokens

Token	Type
p	Identifier
=	Assignment operator
(	Left Parenthesis
a	Identifier
*	Multiplication operator
b	Identifier
)	Right Parenthesis
+	Addition operator
(	Left Parenthesis
b	Identifier
*	Multiplication operator
(	Identifier
)	Right Parenthesis
*	Multiplication operator
2	Integer constant

Total tokens = 15

b) Syntax Tree



c) Three Address Code

$$t_1 = a * b$$

$$t_2 = b * c$$

$$t_3 = t_2 * 2$$

$$t_4 = t_1 + t_3$$

$$p = t_4$$

d) Optimized Intermediate Code

$$t_1 = a * b$$

$$t_2 = b * c$$

$$t_3 = t_2 * 2$$

$$p = t_1 + t_3$$

e) Significance of Target Code

- $\Rightarrow$ Produces machine/assembly code.
- $\Rightarrow$ Executed by the processor
- $\Rightarrow$ Optimizes speed & memory usage
- $\Rightarrow$ Final output of a compiler.

Regular Expressions

a) $(\epsilon + aa)^*$

- ϵ
- aa
- $aaaa$
- $aaaaaaaa$
- $aaaaaaaaaa$

b) $(a + b^*)^* abb$

- abb
- $aabb$
- $babbb$
- $bbabbb$
- $aaabbb$

c) $(a + b)^* aba(a + b)^*$

- aba
- $aaba$
- $abab$
- $abbaa$
- $babab$

d) $(a + ab)^*$

- a
- aa
- ab
- aab
- $abab$

e) $(abab + ab)^*$

- $abab$
- $abababab$
- $ababababab$

- abaa b

- abaa baba b

3) i). Number of Tokens

Program:

main()

{

int a=10;

char b="abc";

int c=30;

char d="xyz";

/* comment */

float m=10.5;

}

Tokens:

(

)

{

int

a

=

10

;

char

b

=

"abc"

;

int

c

=

30

;

"xyz"

;

{

m

=

40.5

;

}

Total tokens = 28

ii) Lexical errors

```
void main()
```

```
{
```

```
int x=10, y=20;
```

```
char *a;
```

```
a = &x
```

```
x = 1xab;
```

```
}
```

Errors: $a = \&x;$ → assigning address of integer to character pointer

• $1xab$ → invalid numeric literal.

~~7~~

4) Regular Expressions:

i) Even number of 0's

$$1^* (01^* 01^*)^*$$

ii) At least one 1

$$0^* 1 0^*$$

(iii)

$$(0+1)^* 1 (0+1)^*$$

iii) odd number of 0's & odd number of 1's

$$(00+11)^* (01+10) (00+11)^*$$

iv) starts with 1 & ends with 0

$$1 (0+1)^* 0$$

v) NO consecutive 1's

$$(0+10)^* ?$$

5) Lexical Analyzer

:/p

int var 123 = 12345;

float var 456 = 123.45;

if (var 123 == var 456)


```

int
var 123
=
12345
;
float
var 456
=
123.45
;
if
{
var 123
==
var 456
}
return
;

```

Total tokens = 19

b) Tokens by category

key words	Identifiers	operators	Separators
• int	• var 123	• =	• ;
• float	• var 456	• =	• ;
• if	• var 123	• ==	• (
• return	• var 456		•)
			• {
			• }
Integer Literals			
• 12345			
		Floating Literals	
		• 123.45	

c) FSM State Transitions

Token type	Approx Transition
Keywords	17
identifiers	24
operators	4
separators	7
Integer (literal)	5
Floating literal	6

{ Total FSM
Transition ≈ 63 }

Reason:

* Each character causes one state transition keywords & identifiers require transition for every letter/digit.

* Floating literals require extra transitions for the decimal point.